

UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

COMPUTER AND SOFTWARE ENGINEERING DEPARTMENT

NYARUGENGE CAMPUS

- **Module Code/Title:** WEB DESIGN
 - **Project Title:** MediConnect: A Community Healthcare Appointment Request Demo
 - **Prepared By:** NKUSI NGENZI Chaste
 - **Registration Number:** 225056357
 - **Year of Study:** 2025-2026
 - **Academic Year:** 2026
 - **Submission Date:** July 19, 2026
-

Table of Contents

1. Chapter 1: Introduction

- 1.1 Historical Background of the Case Study
- 1.2 Problem Statement
- 1.3 Objectives of the Project
- 1.4 Expected Outcomes
- 1.5 Project Rationale (Importance to the Community)

2. Chapter 2: System Analysis and Design

- 2.1 System Analysis
 - 2.1.1 Intended Users of the Project
 - 2.1.2 Intended System Partners
- 2.2 System Design
 - 2.2.1 High-Level Architecture
 - 2.2.2 UI Design & Flowcharts
 - 2.2.3 Data / Design Structure

3. Chapter 3: Implementation

- 3.1 Introduction
- 3.2 Homepage (index.html) Overview
- 3.3 Appointment Form (appointment.html) & Form Upgrades Overview
- 3.4 Frontend Logic & Validation (script.js)
- 3.5 Styling & Design System (style.css)

4. Chapter 4: Conclusion and Recommendation

- 4.1 State of the Implemented System

- 4.2 Recommendations

- 4.2.1 To the Lecturer (Technicalities)

- 4.2.2 To the School (Time Constraints)

- 4.2.3 To the Partners (Clinic Owners)

- 4.2.4 To the Users (Training)

- 4.2.5 To the Beneficiaries (Patients)

5. **Appendix**

- Project Repository & Student Metadata

Chapter 1. Introduction

1.1. Historical Background of the Case Study

In many developing nations, local communities rely heavily on regional healthcare clinics and hospitals for primary care. In Rwanda, community health centers manage high volumes of patients daily. Historically, scheduling appointments at these facilities has been a manual, paper-based process. Patients walk to the clinic early in the morning and queue for hours, sometimes only to find out that the clinic has run out of slots for the day.

With the rapid expansion of mobile network coverage and internet access across Rwanda, there is a tremendous opportunity to leverage lightweight web technologies. The **MediConnect** project represents a frontend model designed to show how a simple, account-free, fast web application can bridge the gap between community clinics and patients by streamlining the initial appointment request stage.

1.2. Problem Statement

The manual scheduling bottleneck in local clinics results in several key issues:

1. **Inefficient Wait Times:** Patients spend excessive time in physical queues, causing loss of productivity and long queues.
2. **Health Risks:** Overcrowded waiting rooms increase the risk of cross-contamination among patients with communicable diseases.
3. **Administrative Overload:** Clinic staff face sudden rushes of patients without prior knowledge of daily case loads, complicating staffing and supply preparations.
4. **Accessibility Barriers:** Existing appointment booking portals are often heavy, require complex multi-step account registrations, and demand high-speed broadband connections which are inaccessible to users with budget-friendly mobile devices.

1.3. Objectives of the Project

General Objective

To design and implement a responsive, lightweight, accessible frontend web application (**MediConnect**) that allows patients to request routine community healthcare appointments without requiring account creation.

Specific Objectives

- **Deliver Clean Structure:** Build semantic and fully accessible HTML5 web templates for both home navigation and request submissions.
- **Establish Responsive CSS Design System:** Define global style sheets using CSS custom properties, grid layouts, and flexbox configurations for fluid rendering across any device viewport.
- **Implement Client-Side Validation:** Utilize JavaScript to restrict request inputs (such as preventing booking past dates and validating phone number ranges).
- **Upgrade Form UX:** Display contextual inline validation alerts and present a detailed glassmorphic success confirmation modal upon booking completion.

1.4. Expected Outcomes

- An intuitive landing page detailing instructions, routine booking call-to-actions, and emergency support advice.
- A booking page containing a functional appointment form with real-time feedback states.
- Fully responsive layout support operating seamlessly from small smartphones up to desktop screens.
- Sub-second page load times achieved by relying entirely on optimized vanilla code with zero external library dependencies.

1.5. Project Rationale (Importance to the Community)

MediConnect provides a direct benefit to local communities:

- **For Patients:** Reduces wasted travel and wait times, enabling them to book routine appointments from their phones.
 - **For Health Centers:** Enables structured scheduling lists to help staff prep equipment and manage daily shifts.
 - **For Public Health:** Keeps stable patients out of emergency clinics, freeing up resources for critical cases.
-

Chapter 2. System Analysis and Design

2.1. System Analysis

2.1.1 Intended Users of the Project

- **General Public / Patients:** Individuals seeking routine consultations, child checkups, vaccinations, maternal care, or follow-up visits.
- **Vulnerable Groups:** Elders or family carers who need a simple interface with large, readable fonts and straightforward forms.

2.1.2 Intended System Partners

- **Community Clinics & Health Centers:** Local facilities responsible for reviewing requests and contacting users to confirm their slots.
- **University Web Development Laboratories:** Academic review panels verifying clean coding standards and interface accessibility rules.

2.2. System Design

2.2.1 High-Level Architecture

The project is built as a lightweight client-side application. The browser requests static assets from the repository server and processes all logic locally:

```
graph TD
```

```
Client[Patient's Browser] -->|Requests Files| Server[Static Web Server]
```

```
Server -->|Serves index.html, style.css, script.js| Client
```

```
Client -->|Validates Inputs| JS[script.js Engine]
```



```
JS -->|Passes Validation| Modal[Dynamic Success Modal]
```

2.2.2 UI Design & Flowcharts

The workflow for a user requesting an appointment is mapped below:

```
graph TD
```

```
Start([User Lands on Homepage]) --> ViewSteps[Read Three-Step Instructions]
```

```
ViewSteps --> ClickBook[Click Book Appointment]
```

```
ClickBook --> FillForm[Fill out appointmentForm]
```

```
FillForm --> Validate{JavaScript Validation Passed?}
```

```
Validate -->|No| ShowErrors[Highlight Invalid Fields with Inline Error Text]
```

```
ShowErrors --> FillForm
```

```
Validate -->|Yes| ShowModal[Display Dynamic Glassmorphic Success Modal]
```

```
ShowModal --> ResetForm[Reset Form & Input Statuses]
```

2.2.3 Data / Design Structure

The layout styles and theme colors are managed through a unified CSS variable architecture:

- **Primary Brand Accent (``--primary``):** Teal ``#087f8c`` - represents clinical precision and professional care.
 - **Muted / Text Secondary (``--muted``):** Slate ``#5d707c`` - delivers readable, low-fatigue body text.
 - **Core Contrast Background (``--mint``):** Tint ``#e9f7f5`` - creates a clean and comforting aesthetic.
 - **Typography:** ``Inter`` fallback font-stack for high-legibility.
-

Chapter 3. Implementation

3.1. Introduction of the Section

The implementation phase of MediConnect focuses on delivering semantic markup, lightweight styles, and interactive logic. Below is an overview of the core file implementation.

3.2. Homepage (index.html) Overview

The landing page [index.html](#) introduces the app's functionality:

- **Navigation Header:** Branding with a custom vector medical icon and links to navigation sections.
- **Hero Section:** Clear copy outlining the community healthcare approach and call-to-action buttons.
- **Step-by-Step Guide:** Shows a three-stage instruction track to guide users.
- **Emergency Alert:** Contains a warnings banner advising users experiencing life-threatening symptoms to call emergency services.

3.3. Appointment Form (appointment.html) Overview

The page [appointment.html](#) holds the appointment booking form:

- **Form Grid:** Inputs for Name, Phone, Facility Name, Service Type, Preferred Date, Preferred Time, and Reason for the visit.
- **Validation Spans:** Empty containers (``) are placed contextually below each input to show error text when needed.
- **Success Modal Dialog:** A hidden container (``#successModal``) is positioned at the bottom of the body, which dynamically displays booking details once input validation succeeds.

3.4. Frontend Logic & Validation (script.js)

The script file [script.js](#) manages client-side interactivity:

- **Date Bounds:** Restricts the date picker's minimum attribute to the current calendar date (``dateInput.min = new Date().toISOString().split("T")[0]``).
- **Real-Time Validation:** Input and blur event listeners evaluate inputs locally.
- **Custom Validations:**

- Name inputs must be at least 3 characters long.
- Phone inputs must containing at least 8 digits.
- Reason descriptions must contain at least 10 characters.
- **Modal Triggering:** Once all criteria pass, the form captures the values, populates the modal summary spans, displays the modal, and resets the form.

3.5. Styling & Design System (style.css)

The stylesheet [style.css](#) styles layouts and responsive behaviors:

- **Viewport Adaptability:** Uses CSS grid template changes at `800px` and `520px` to format inputs in single columns on mobile viewports.
 - **Visual States:** Employs soft red borders (`#fa5252`) and background fills (`#fff5f5`) to flag input errors, and soft teal borders (`#087f8c`) to mark valid fields.
 - **Animations:** Implements `@keyframes` rules to scale and fade transitions, creating a premium feel.
-

Chapter 4. Conclusion and Recommendation

4.1 State of the Implemented System

The frontend structure of **MediConnect** is 100% complete. The landing grid and appointment booking form are fully functional as a frontend demonstration, showing how local validation can improve usability.

4.2 Recommendations

4.2.1 To the Lecturer (Technicalities)

The vanilla coding method used in this project demonstrates high performance and layout speed. For future modules, it is recommended to introduce modern build tools (such as Vite) and component frameworks to show students how these static pages can scale.

4.2.2 To the School (Time Constraints)

Recommend extending laboratory project deadlines and curriculum modules to include backend services (like Node.js or Python Flask) and database integrations, allowing students to transition frontend layouts into full-stack applications.

4.2.3 To the Partners (Clinic Owners)

Community health clinics should consider adopting lightweight web systems to collect appointment requests, helping staff organize daily operations and prepare clinic rooms.

4.2.4 To the Users (Training)

Because the platform requires no registration credentials, user training is minimal. Clinics can print simple QR code notices at desks to introduce patients to online bookings.

4.2.5 To the Beneficiaries (Patients)

Patients are encouraged to use the portal to schedule routine checkups ahead of time, reserving emergency channels for critical conditions.

Appendix

Project Repository & Student Metadata

- **Student Name:** NKUSI NGENZI Chaste
- **Registration Number:** 225056357
- **Public GitHub Link:** [nkusingenzi-tech/assignment](#)
- **Local Workspace Root:** [mediconnect-school-project](#)